Technical Architecture & Implementation

.NET solution design, data model, APIs, AI pipelines, security, DevOps and testing

ConstructionFlow AI | Kosovo-first, multilingual SaaS | Version 1.0

Document purpose

A practical planning reference. All costs, dates, market assumptions, pricing and estimates are illustrative and must be validated with local customers, engineering specialists, legal advisers and actual vendor quotes.

Contents

1. Architecture principles and boundaries

2. System context and request flow

3. Technology selection

4. Solution and module layout

5. Data model and tenancy

6. API design and examples

7. AI workflows, prompts and safeguards

8. State machines and asynchronous reliability

9. Security, privacy and audit

10. DevOps, infrastructure and observability

11. Testing strategy and AI evaluation

12. Delivery backlog and engineering sequence

1. Architecture principles and boundaries

Use a modular monolith in ASP.NET Core, with explicit application/domain/infrastructure boundaries. AI is an assistive subsystem: all prices, totals, approvals and permissions remain deterministic application logic.

• Modules: Identity & Tenancy; Projects; Budgeting; Procurement; Invoices; Site Reporting; Documents; AI Orchestration; Notifications; Audit & Reporting.

• Communication: in-process domain events and transactional outbox for reliable asynchronous work; avoid microservices until scaling or team boundaries justify them.

• Frontend: React + TypeScript PWA with server-driven authorization checks; mobile-first reporting forms.

2. System context and request flow

User -> React PWA -> ASP.NET Core API -> domain/application services -> PostgreSQL. Files are uploaded using short-lived signed URLs to object storage; background workers handle scanning, extraction, indexing and AI jobs.

• Document ingestion: upload -> malware scan -> object storage -> metadata DB -> OCR worker -> review queue -> optional embeddings -> searchable index.

• RFQ comparison: upload offers -> extraction -> normalization -> validation -> deterministic comparison -> human approval -> PO.

• Voice reporting: upload audio -> transcription -> structured JSON proposal -> field validation -> engineer correction -> approval -> tasks.

• RAG: authorization filters -> hybrid search -> bounded context -> LLM answer -> citation verification -> audit trace.

3. Technology selection

Layer	Recommended technology	Reason
Backend	ASP.NET Core / C#	Strong domain workflows and validation
UI	React + TypeScript	Responsive browser and PWA
Database	PostgreSQL + EF Core	Relational integrity, transactions
Search	pgvector + PostgreSQL FTS initially	Lower operational complexity
Storage	S3-compatible or Azure Blob	Durable source files
Background	Hangfire or hosted workers + queue	Retries and async processing
LLM	Provider abstraction over Azure OpenAI / alternatives	Swap models and control spend
OCR	Document Intelligence or evaluated alternative	Invoice and quotation extraction
Speech	Evaluated multilingual speech service	Test Albanian construction terminology
Telemetry	OpenTelemetry + structured logs	Tracing, metrics and errors

4. Solution and module layout

Suggested solution: src/Api; src/Modules/Identity; Projects; Procurement; Finance; SiteReports; Documents; AI; SharedKernel; Infrastructure; Workers; web/; tests/Unit; Integration; Contract; E2E. Within each module use Domain, Application, Infrastructure and Contracts folders.

- Use explicit command/query handlers or application services; CQRS is optional, not required.
- Expose DTOs, not EF entities. Validate requests at the boundary and enforce invariants in the domain.
- Separate AI extraction schemas from approved business records to prevent unreviewed writes.

5. Data model and tenancy

Entity	Important fields / relationships
Organization	Id, Name, Locale, Subscription
Membership	OrganizationId, UserId, Role, Status
Project	OrganizationId, Code, ManagerId, Status
BudgetVersion	ProjectId, Version, ApprovedBy, ApprovedAt
Supplier / RFQ / Quote	OrgId, RFQId, currency, terms, line items
PurchaseOrder	QuoteId, approval status, approved version
Invoice / DeliveryReceipt	SupplierId, POId, documentId, reviewed totals
DailySiteReport	ProjectId, Date, AuthorId, approval status
Document	OrgId, ProjectId, classification, storage key, hash
AIJob / Extraction	Provider, model, status, confidence, reviewer
AuditEvent	Actor, tenant, action, entity, timestamp

• Enforce OrganizationId on all tenant-scoped queries and foreign-key relationships; add row-level security where appropriate as defense in depth.

• Use optimistic concurrency tokens for approvals and financial edits; soft deletion only where business rules justify it.

6. API design and examples

Method / path	Purpose
POST /api/v1/projects	Create project
GET /api/v1/projects/{id}/summary	Read scoped dashboard
POST /api/v1/rfqs	Create RFQ
POST /api/v1/rfqs/{id}/quotes	Attach supplier quotation
POST /api/v1/quotes/{id}/compare	Run normalized comparison
POST /api/v1/purchase-orders/{id}/approve	Approve with version check
POST /api/v1/documents/uploads	Create signed upload request
GET /api/v1/ai-jobs/{id}	Poll async status
POST /api/v1/site-reports/drafts	Create/edit site draft
POST /api/v1/projects/{id}/ask	Authorized RAG question

• All endpoints use tenant-scoped authorization, consistent ProblemDetails errors, pagination, filtering, idempotency keys for financial commands and request correlation IDs.

• Example async result: HTTP 202 with jobId and statusUrl; worker transitions queued -> processing -> needs_review -> completed/failed.

7. AI workflows, prompts and safeguards

Invoice/quote extraction

• OCR output -> document-type classifier -> schema-constrained extraction -> deterministic arithmetic validation -> confidence flags -> reviewer edits -> approved record.

• Prompt instructions must treat uploaded document text as untrusted; ignore embedded commands and never execute instructions from supplier documents.

Voice-to-report

• Speech recognition -> domain dictionary normalization -> structured field proposal -> evidence links to transcript -> human confirmation.

RAG

• Chunk by document structure, preserve page offsets and versions, apply ACL before retrieval, combine lexical and vector search, cite retrieved passages.

• Use evaluation sets for Albanian terms, supplier formats, scanned documents, hallucinations, cross-tenant leakage and adversarial prompt injection.

• Never use an LLM as the authoritative calculator for financial totals or quantities.

8. State machines and asynchronous reliability

Aggregate	States
Document	Uploaded -> Scanning -> Extracting -> NeedsReview -> Approved / Rejected
AIJob	Queued -> Running -> Completed / Failed / Cancelled
RFQ	Draft -> Issued -> OffersReceived -> Evaluated -> Awarded / Closed
PurchaseOrder	Draft -> PendingApproval -> Approved / Rejected -> Sent
DailyReport	Draft -> Submitted -> Approved / Returned

• Implement retries with exponential backoff, poison-job quarantine, dead-letter visibility and idempotent worker handlers.

• Use an outbox table for events that must be published after database commit; avoid distributed transactions.

• Preserve immutable source documents and version extracted fields; track who corrected each field.

9. Security, privacy and audit

• OIDC/OAuth2 login, MFA for privileged users, per-project role checks and organization-scoped authorization.

• Encrypt data in transit and at rest; secrets in a managed secret store; signed URLs expire quickly.

• Malware scanning, MIME validation, file size limits, upload rate limits and safe document rendering.

• Avoid placing raw personal data or complete contracts in application logs or telemetry.

• Support subject access/export/deletion workflows subject to applicable retention duties; verify Kosovo law with counsel.

• For AI vendors: verify data processing terms, region, retention, subprocessors and model-training defaults before production.

• Threat model cross-tenant access, malicious PDFs, prompt injection, forged approvals, replayed webhooks and compromised site devices.

10. DevOps, infrastructure and observability

• Development: Docker Compose for PostgreSQL, object-storage emulator and worker; seeded synthetic demo data.

• CI: lint, compile, unit tests, integration tests, dependency scan, secret scan and migration checks.

• CD: staging deployment, smoke tests, manual production approval, reversible rollout and migration plan.

• Infrastructure as code: Bicep if Azure is selected, or Terraform for multi-cloud requirements.

• Monitoring: API latency/error rate, job backlog, OCR failures, AI cost per tenant, token usage, document processing time and approval latency.

• Backups: daily database backups with point-in-time recovery where supported, storage versioning, monthly restore drill and documented RPO/RTO objectives.

11. Testing strategy and AI evaluation

Test layer	Examples
Unit	Price totals, VAT handling, state transitions, permissions
Integration	EF transactions, outbox, storage and worker retries
Contract	OCR/LLM provider adapter schemas, API versioning
E2E	RFQ to PO, invoice matching, voice report review
Security	Tenant isolation, upload abuse, prompt injection
AI evaluation	Field extraction F1, transcript correction, citation precision

• Maintain a versioned golden dataset with consented or synthetic Albanian invoices, quotations, site notes and contracts.

• Gate AI rollout on quality by document type, not on one global accuracy figure; record human corrections as evaluation labels.

12. Delivery backlog and engineering sequence

Sprint	Engineering deliverables
1	Repo, auth, tenancy, project CRUD, CI, base migrations
2	Budgets, suppliers, RFQs, quotes, deterministic totals
3	File storage, scan pipeline, OCR jobs, review UI
4	Purchase approvals, invoice matching, audit trail
5	Site report CRUD, speech pilot, task generation
6	RAG retrieval with ACL and citations, dashboards
7	Security hardening, tests, observability, backup restore
8	Pilot deployment, feedback, cost profiling, fixes

• Treat each sprint as a planning unit, not a fixed one-week promise. A solo developer may need 12-20+ weeks for a dependable pilot.

• Release gate: zero critical security issues, completed UAT workflows, demonstrated restore, documented manual fallbacks and monitored AI spend.